ENGINEERING AND SECURITY REVIEW · 14 OCTOBER 2026

Custodian project audit and repair plan

Review date: 14 October 2026
Intended product: An offline Windows assistant that finds and understands files, performs approved PC tasks precisely, and protects private data and the operating system.
Decision: Continue development, but treat the current build as a prototype. Fix isolation, recovery, truthful results, and installation before expanding automation.

Extended review: Section 16 adds findings F40–F46 after comparing the owner's original design documents and investigating stale answers, whole-system search, missing file operations and side-by-side execution. The companion implementation roadmap is `D:\Custodian\docs\CUSTODIAN_OFFLINE_ASSISTANT_IMPLEMENTATION_ROADMAP.md` (also available as HTML). It contains detailed target behavior, eight delivery milestones after the baseline milestone, 38 proposed implementation tickets, an original-build-plan mapping and 40 additional proposed acceptance cases. These are recommendations, not implemented fixes.

Open the companion implementation roadmap.

1 Executive assessment

Custodian has a credible foundation: a working desktop interface, local model integration, a restricted tool vocabulary, deterministic Rust tool validation, file discovery, duplicate detection, signed cleanup approvals, and basic undo. It is not yet the general offline assistant described in the product goal. It is primarily a desktop companion with a limited set of file, application, reminder, and appearance-setting actions.

The main problem is not simply model quality. The repository contains several partially separate architectures. The desktop runs tasks directly through `assistant/mod.rs` and `custodian-core`; the engine, persistence, journal, workflow service, parser, and broker are only partly connected or remain standalone building blocks. Security primitives exist, but their presence does not prove that every live task uses them.

Five issues deserve immediate attention:

1. The AI sandbox is **opt-in in the current code**, despite documentation describing it as the normal path. If sandbox initialization fails, the desktop also starts the AI as an ordinary user process.
2. The desktop cleanup implementation moves files before persisting their recovery entry. A crash or write failure in that gap can leave a moved file absent from the manifest. Undo and purge have similar ordering gaps.

3. Cleanup manifests are plain editable JSON and contain paths used for restore and purge. Hashes of file content do not authenticate these paths or authorize the operation.
4. Several unfinished components report successful or safe outcomes without doing the work. A dormant offload operation actually overwrites the original contents while uploading nothing.
5. There is no complete clean-machine offline installation and acceptance gate. Passing unit tests does not establish that the installed app can start its model, safely execute tasks, recover after crashes, or protect private files.

The right direction is a **bounded assistant with a single trustworthy execution service**. The model proposes typed actions; policy checks access and risk; approvals bind to exact plans; execution records recovery information before changes; deterministic verification supplies the final result. Broaden the supported task catalogue through tested adapters, rather than giving the model arbitrary shell or administrator access.

2 Scope and interpretation of evidence

This review examined the working tree, including existing uncommitted modifications. The source was not reset, and this audit does not modify product behavior. Findings reference repository paths and function names, with line numbers for selected priority issues. Paths are relative to `D:\Custodian` unless stated otherwise.

Evidence labels used below:

Label	Meaning
Confirmed source defect	The current implementation demonstrates the behavior or missing guard. Some harmful consequences still require a targeted runtime reproduction.
Observed check result	A command was executed during this review and its outcome was inspected.
Capability gap	A requested capability is missing, a placeholder, or not connected to the desktop.
Risk requiring validation	The code suggests a failure scenario, but this review did not demonstrate an exploit or reproduce the full scenario.
Proposed design	A recommendation that must be approved as a project decision before implementation if it changes the canonical design.

This is a broad engineering audit, not a claim that every possible defect has been found. It does not establish the absence of vulnerabilities, certify an installer, or benchmark the models. Real-desktop approval interactions, adversarial filesystem races, clean-machine installation, PostgreSQL migrations, live model evaluation, antivirus integration, and power-loss recovery remain explicit validation tasks.

3 What we should be building

The product should turn a request such as “find my latest invoice, tell me its size, and move a copy into my accounts folder” into a transparent sequence:

1. Identify the request and the user-approved search roots.
2. Search metadata locally; distinguish similarly named files with full paths, size, and dates.
3. Ask the user to choose if the target is ambiguous.
4. Prepare a typed plan with source identity, destination, expected effects, and recovery method.
5. Obtain consent appropriate to the risk and bind it to that plan.
6. Record the plan durably before writing or moving anything.
7. Execute using restricted filesystem or OS APIs.
8. Verify the actual result and report exactly what happened.
9. Offer undo when the operation supports it; disclose irreversible effects before execution.

The security goal has two different parts. First, prevent Custodian itself from leaking data or damaging the PC. Second, help the user inspect the PC's existing protection and handle suspicious content safely. The second part should integrate with supported Windows protection mechanisms; an LLM that reads filenames is not an antivirus engine.

“All tasks” needs a capability roadmap. A safe assistant can eventually support many tasks, but some require explicit administrator consent, application-specific APIs, network access, or human intervention. The interface must distinguish supported, unsupported, unavailable, permission-denied, ambiguous, and failed tasks. Unsupported work should produce an explanation or a safe handoff, never a fabricated success.

3.1 Capability coverage

User expectation	Current state	Required completion
Find files locally	Name matching in six known user folders	Approved additional drives and folders, full result paths, filters, candidate selection, indexing

User expectation	Current state	Required completion
Tell me a file's size and details	Scanner collects size; largest-file tool reports sizes	A dedicated selected-file metadata tool and rich file results
Understand documents	Parser reads bytes as lossy text; excerpt pipeline is not wired	Bounded parsers for supported formats, private-content permissions, optional local semantic search
Organize and edit files	Duplicate quarantine and old-temp cleanup	Copy, move, rename, create, and supported edits through the transaction service
Change settings	Six per-user registry settings	Typed adapters for additional settings, exact snapshots, conflict-aware undo, verified postconditions
Control applications	Open known or Start-menu apps, list windows, request close	Stable app/window identities, task-specific adapters, truthful completion and timeout handling
Reminders and task list	Sealed local storage; app-running scheduler	Reliable startup, resume and missed-task behavior; recurrence if required
Work offline after installation	Model files exist in this checkout; runtime relies on development files	Bundled runtime, native libraries, model provisioning, offline clean-machine tests
Protect private files	Restricted model tools and optional AppContainer	Explicit access policy, enforced isolation, protected roots, authenticated recovery metadata
Safeguard against external threats	Download/signature/ML helpers include placeholders	Supported scanner and trust checks with unknown/error outcomes; no autonomous LLM deletion
Execute quickly and precisely	Rules avoid inference for familiar requests	Cancellation deadlines, indexing, resource budgets, latency measurements, stronger acceptance gates

4 Current architecture and why the build feels unreliable

The live desktop route is:

```
React chat and cards
    -> Tauri commands
    -> Rust intent rules
        -> direct Rust tools for recognised requests
        -> Node local model for other requests
            -> authenticated tool-request frames
            -> Rust revalidation
            -> direct tools or pending cards
    -> cleanup manifests / settings history / task list
    -> history ledger recorded after selected operations
```

Important distinctions:

- The model does have tool definitions and the desktop handles its calls. `HONESTY.md` still contains an older sentence saying the agent has no tool execution/search; later paragraphs describe tool calls correctly. The current implementation is more capable than that older sentence implies.
- The desktop crate does not depend on `custodian-engine` or `custodian-store`. Its active cleanup is `custodian-core::cleanup::Quarantine`, not the engine's quarantine implementation.
- `ApprovalExecutor` in the engine verifies approvals and inserts a transaction record; it is not a complete filesystem operation executor. Its database insertion error is discarded.
- The journal and ledger are useful libraries. The journal is not the desktop cleanup transaction coordinator.
- `TransactionRecoverer` is a state-transition helper using a supplied callback. It does not reconstruct real filesystem transactions and database mirrors from disk at startup.
- The orchestrator is a Temporal worker, but its activities log IPC rather than send it. Browser actions validate names and return success strings without driving a browser.
- The ML host is separate from the chat model. Its embeddings, voice, and predictive helpers are placeholders; the desktop does not become a semantic search engine merely because vector tables exist.

This creates a practical development trap: adding another helper makes the architecture look more complete without improving the path the user actually exercises. Every new feature should have a demonstrated route from the installed UI to policy, execution, verification, recovery, and history.

5 Verification performed

The following outcomes were observed during this review. Initial failures and successful reruns are deliberately separated.

Check	Observed outcome	Interpretation
`npm run typecheck`	Passed	TypeScript projects and desktop type checks compile
`npm run lint`	Passed	JavaScript and TypeScript lint rules pass
`npm run build -w apps/desktop`	Passed	Vite produced the frontend bundle; this is not an installer test
`npm test` with default review temporary directory	Failed imports with temporary SSR cache `ENOENT` errors	Test-environment/cache issue; not evidence of failed assistant logic
`npm test` with `TEMP` and `TMP` set to `D:\Custodian\target\audit-temp`	Passed: desktop 21, agent 57, broker 7 tests	85 JavaScript tests passed after environment adjustment
`python -m pytest -q`	Passed: 17 tests	ML host and IPC tests pass; model accuracy is not covered by this result
`python -m ruff check services/mlhost tests training`	Passed	Selected Python source lint passes
`python -m mypy services/mlhost tests training`	Passed: 17 source files	Selected Python typing checks pass
`python tests/security/scan_secrets.py`	0 findings	This scanner found no matching secrets; not a full secret-history audit
`cargo test --workspace --offline`	Blocked compiling desktop: `OUT_DIR env var is not set` at `tauri::generate_context!()`	Build failure observed; cause not conclusively attributed to source because build.rs exists and this environment has path restrictions
Rust workspace excluding desktop	Blocked by migration-directory canonicalization: access denied in `sqlx::migrate!`	Environment restriction prevented complete engine/store verification
Selected Rust crates with workspace-local temporary folder	Approval tests passed 7; core passed 42 of 43, temp deletion test failed with zero deletions	Requires comparison with default temp folder; `SafePath` canonicalization is sensitive to this environment
Final full Rust workspace outside restricted sandbox	Passed: 263 tests, 3 explicitly ignored	Earlier Rust failures were environment-dependent; live-model and Start-menu integration tests remained ignored

The final full Rust workspace rerun outside the restricted review sandbox passed. The earlier Rust failures were environment-dependent and must not be counted as established product failures. Full details are in the verification addendum at the end of this document.

The repository's `HONESTY.md` reports a prior model evaluation of 49/57 cases, approximately 86 percent, from 12 October 2026. That is historical evidence, not a fresh measurement from this review. The evaluation uses canned desktop results and therefore cannot establish end-to-end task accuracy. This checkout contains both a roughly 1.1 GB Qwen2.5 model and a roughly 2.5 GB Qwen3 model, but their presence does not prove correct loading inside the installed application or sandbox.

6 Detailed findings and corrective actions

Priorities: **P0** means stop exposing the affected behavior until corrected; **P1** means required for a trustworthy offline release; **P2** means important capability or maintainability work after the safety foundation; **P3** means later roadmap work. A P0 on a dormant component does not mean the live desktop currently calls it.

F01 The AI sandbox is opt-in and falls back to an unrestricted process

Priority: P0. **Evidence:** Confirmed source defect. **Location:** `apps/desktop/src-tauri/src/assistant/mod.rs:1673`, especially line 1676; `ensure_agent` at line 1566.

`spawn_sandboxed_agent` returns `None` unless `CUSTODIAN_AGENT_SANDBOX` equals `1`. An unset variable therefore skips the AppContainer. `ensure_agent` then launches `AgentProcess::spawn`, which runs under the normal user context. A sandbox startup or model-readiness failure takes the same fallback route and disables further sandbox attempts for the process lifetime. README and HONESTY describe `0` as an opt-out, which does not match this condition.

Impact: The ordinary agent process has the user's normal file/network access even though tool-call validation restricts what the model can request through Rust. Process permissions and tool permissions are separate controls. Native model libraries, dependencies, or compromised agent code are outside the tool allow-list protection.

Fix: Make AppContainer startup the release default. If it fails, keep deterministic tools available and report AI unavailable with an actionable diagnostic. Allow unrestricted startup only as a clearly labelled development mode. Inspect the actual child token and network/file access, rather than treating an environment flag as proof of isolation.

Acceptance: On a clean PC with no special environment variables, the child cannot read a protected fixture outside grants, write into readable resources, or reach a test network endpoint. Forced sandbox failure never starts a plain child.

F02 Resource limits fail open and normal startup has a race

Priority: P1. **Evidence:** Confirmed source behavior. **Location:** `apps/desktop/src-tauri/src/agent.rs`, `agent_job` and `AgentProcess::spawn`.

The plain child starts before job assignment. If job creation or assignment fails, `agent_job` logs a warning and returns `None`, allowing the child to continue without the intended limits. The normal spawn also inherits the parent environment, unlike a deliberately minimal environment.

Fix: Use suspended startup and assign the job before resuming on every permitted path. Fail startup if essential containment cannot be applied. Pass only required environment values, trusted absolute executable paths, and package paths. Expose containment state in diagnostics.

Acceptance: An assignment failure leaves no running agent. Child processes exit when the application shuts down. Tests verify memory limits and that sensitive inherited environment variables are absent.

F03 Desktop cleanup recovery entries are persisted after mutations

Priority: P0. **Evidence:** Confirmed source defect. **Location:** `crates/custodian-core/src/cleanup.rs:457`, move at line 505, manifest write at line 520; undo around lines 534–573; purge around lines 617–630.

The comment says persisting after every move prevents losing track, but a crash between the move and manifest write still loses the entry. `write_manifest` writes a temporary JSON file and renames it without explicitly flushing its contents. Undo first moves a payload out and only then records `restored_to`; purge deletes payloads before updating their state. A write failure can leave files changed while the caller receives an error.

Fix: Persist an intent containing original, destination, identities, digest, and recovery strategy before each mutation. Flush it, perform the operation, verify the result, then persist completion. Startup must reconcile incomplete intents against actual file identities. Use a single coordinator and failure injection at every boundary. Do not assume replacing the desktop route with today's engine automatically completes this work.

Acceptance: Kill the process before and after every journal write, move, copy, restore, and deletion. Restart always discovers every payload and presents a correct recoverable state.

F04 Cleanup manifests are unauthenticated authority-bearing input

Priority: P0. **Evidence:** Confirmed source design flaw; exploitability requires targeted testing. **Location:** `cleanup.rs`, `ManifestEntry`, `read_manifest`, `manifests`, `undo`, and `purge`.

The desktop stores plain JSON containing `original`, `stored`, `keep`, hashes, and dates. Restore and purge later trust those paths. Content hashes can be recomputed by someone editing the manifest; they do not authenticate its provenance. The signed cleanup approval protects the initial plan, not subsequent

manifest edits. The root also lacks an explicit owner-only ACL and reparse-point enforcement, as acknowledged in HONESTY.

Fix: Authenticate and version the entire manifest, enforce the vault root and permitted destination roots on every read, and bind entries to file IDs and transaction IDs. Validate plan IDs as plain identifiers. Require immutable execution records for purge decisions. Quarantine corrupted metadata for manual recovery rather than following it. Separate cleanup retention from any genuinely encrypted private-file vault.

Acceptance: Changing any path, digest, creation date, or plan ID causes a safe refusal. Junctions, substituted roots, and references outside the vault cannot redirect a purge or restore.

F05 Ordinary rename undo does not verify payload content

Priority: P1. **Evidence:** Confirmed source defect. **Location:** `cleanup.rs:534` , `move_verified` ; `crates/custodian-winsec/src/fsops.rs` , `move_no_replace` .

`undo` passes the expected hash to `move_verified` , but `move_no_replace` immediately returns after a successful same-volume rename. The verification callback runs only on cross-volume copies. Consequently the live cleanup implementation can restore a modified same-volume payload without detecting its mismatch. The separate engine `QuarantineOp::inverse` explicitly hashes its payload before moving it, showing that the two routes enforce different guarantees.

Fix: Verify the stored payload against authenticated metadata before restoration and verify the resulting object afterward. Use handle-based identity checks to address replacement races. Preserve a failed payload for investigation rather than silently restoring it.

Acceptance: Modify the bytes of a quarantined fixture on the same volume. Undo refuses to restore it and reports an integrity failure.

F06 Path validation and content checks are separated from the operation

Priority: P1. **Evidence:** Risk requiring validation, supported by source. **Location:** `custodian-winsec::SafePath` , `cleanup.rs` , `temp.rs::still_safe` , and `fsops.rs` .

SafePath documents that its validation is point-in-time. Temp cleanup validates and then calls `remove_file` using the original path. Cleanup hashes source and kept-copy paths before moving the source. A junction, parent, or file can change between these steps. Checking that a kept copy existed earlier does not establish it still exists at mutation time.

Fix: Open and operate on verified handles with appropriate sharing rules; reject reparse points and validate volume/file identity. Check ancestry and policy at execution time, including sync-root restrictions. Serialize overlapping transactions. Document the remaining limitations when another process independently deletes files.

Acceptance: A fixture harness repeatedly swaps files, junctions, and parents between validation and mutation. No operation escapes approved roots or silently acts on a substituted object.

F07 Purge treats cloud size and timestamp as proof of identical content

Priority: P1. **Evidence:** Confirmed source weakness. **Location:** `cleanup.rs:364` , `kept_copy_intact` .

For cloud placeholders, the kept copy is accepted using file size and a modification timestamp rounded to seconds. Equal size/time is not a content-hash proof. This can authorize deletion of the last independently verified local duplicate while the remote content is not available to check.

Fix: Keep the local quarantined copy when the keeper cannot be verified locally. A later online feature may use a trusted provider-specific content identity with explicit consent, but metadata equality alone must not satisfy the purge gate.

Acceptance: Replace the keeper with a same-size cloud placeholder retaining the timestamp. Purge keeps the verified local payload and explains why.

F08 Concurrent cleanup and recovery operations have no common lock

Priority: P1. **Evidence:** Confirmed missing coordination; race consequences require validation. **Location:** desktop `approve_plan` , `undo_tool` , `start_safe_place_keeper` , and `Quarantine` .

A new `Quarantine` value is created for calls; it has no transaction lock. Approval, automatic purge, manual purge, and undo can overlap. `history_ledger::WRITER` serializes ledger appends, not filesystem work or manifest replacement. Concurrent writes to the same fixed manifest temporary name can conflict or overwrite state.

Fix: Introduce one mutation queue with per-transaction/per-resource locks and a cross-process single-instance or store lock. Purge and undo must coordinate on the same state machine. Use unique temporary files and deterministic recovery.

Acceptance: Concurrent undo/purge/cleanup requests never lose metadata, double-execute a mutation, or claim contradictory outcomes.

F09 Offload destroys content despite being a placeholder

Priority: P0 for this API. **Evidence:** Confirmed source defect, currently dormant in the desktop. **Location:** `crates/custodian-engine/src/ops/offload.rs` .

`OffloadOp::execute` ignores the upload URL, reads the length, and overwrites the file with `OFFLOADED: size=...` . Its test asserts that destructive placeholder behavior. This is not a safe stub.

Fix: Return an explicit unsupported error before touching the file. Only enable offload after upload durability, remote identity, encryption where required, retrieval, local stub semantics, and rollback are implemented and verified. Offline mode must never invoke remote offload.

Acceptance: With offload unavailable, invoking the API leaves the original bytes unchanged. Failure at any future upload stage retains a recoverable original.

F10 Other public mutation APIs bypass the intended execution contract

Priority: P1 before integration. **Evidence:** Confirmed source gaps in dormant/lower-level APIs. **Location:** `custodian-core/src/executor.rs` ; `custodian-engine/src/fs.rs` .

`execute_action` exposes permanent deletion without an approval argument. Its quarantine reads an entire file into memory, writes encrypted bytes to a destination based only on the basename using an overwriting write, then removes the original, without a recovery record. Two different sources with the same name can overwrite each other's vault entry. Engine `execute_move` uses ordinary rename; `execute_delete` can recursively remove directories. SafePath proves an allowed location, not permission for a destructive action.

Fix: Make these internal primitives inaccessible from unvalidated entry points, remove obsolete alternatives, require validated operation capabilities, use unique transaction identities, no-replace operations, and durable recovery. Stream large data instead of reading whole files for encryption.

Acceptance: Integration cannot call a write primitive without the coordinator's validated capability. Basename collisions and directory deletes cannot destroy unrelated fixture data.

F11 Integrity checks run after changes and missing history can appear healthy

Priority: P1. **Evidence:** Confirmed source defects. **Location:** `apps/desktop/src-tauri/src/history_ledger.rs` , `record` and `check` ; desktop `record_history` .

Selected tasks mutate first and then record history. A tamper result adds a warning and continues; it does not prevent the completed change. `check` returns intact immediately when `ledger.bin` is absent, even if an anchor still exists. If both files are removed, the current store cannot distinguish deletion from first use without an independent initialization record. An appending record followed by failed anchor save also needs explicit recovery semantics.

Fix: Check security/recovery health before risky work. Detect missing ledger with existing anchor. Persist a first-use marker outside the deletable pair if stronger deletion detection is required. Preserve suspicious history, block risky work until recovery, and distinguish incomplete anchor advancement from tampering.

Acceptance: Delete only the ledger, only the anchor, truncate records, corrupt a middle record, and fail anchor persistence. Status and recovery are accurate, and further risky mutations are blocked when integrity is unresolved.

F12 Security guarantees exceed the same-user protection boundary

Priority: P1 for threat model and claims. **Evidence:** Acknowledged architectural limitation. **Location:** DPAPI key storage, ledger anchor, native approval fallback, HONESTY.

DPAPI protects against some offline-copy and other-account access scenarios, but another process running as the same Windows user can generally use that user's DPAPI context. It must not be described as protection against all malware already running as that user. The native confirmation fallback also lacks the real-click hook, which HONESTY explicitly acknowledges. Signed approvals bind a plan but do not independently prove who requested the signature inside a compromised process.

Fix: Define the threat model explicitly. Use restricted process roles, protected storage permissions, secure/native confirmation appropriate to operation risk, and clear degraded-mode behavior. Consider stronger key/service separation only where the threat model requires it. Do not claim that an owner-only ACL defeats a malicious process with the owner's token.

Acceptance: Security documentation distinguishes untrusted model output, other accounts, offline disk copies, same-user malware, and administrator compromise. Tests check each promised boundary.

F13 Offline policy is not centrally enforced for network handoffs

Priority: P1. **Evidence:** Confirmed architecture gap. **Location:** desktop `needs_card`, `web_search_tool`, `open_website_tool`; `custodian-core/src/online_sm.rs`.

Directly recognised user requests can open a browser search or website without consulting the core online state. Model requests wait for a card, but approved actions also use the normal browser. `go_offline` changes an enum and calls an empty broker termination function. The online mode visual therefore does not establish the absence of network-capable handoffs.

Fix: Define strict offline mode versus explicit external-browser handoff. Put a single policy check before every outgoing payload or network-capable launch. If strict offline mode is active, explain that the task needs a network handoff and obtain a bounded exception or refuse it. Show which text/data will leave the PC.

Acceptance: In offline mode, every network tool is denied or receives a visible scoped exception. Ending the exception closes owned broker work. Existing unrelated browsers are not terminated.

F14 Stop can wait indefinitely for the next agent frame

Priority: P1. **Evidence:** Confirmed source defect. **Location:** `apps/desktop/src-tauri/src/agent.rs:129`, `Connection::chat`, and `Connection::ping`.

The loop performs blocking `read_frame` before checking the cancellation flag. During a stalled model load, native call, or child that remains alive without emitting frames, Stop cannot send cancellation until another frame arrives. Ping has no visible deadline either. GPU detection has a timer, but that does not bound all startup and inference stages.

Fix: Use an independent reader and cancellation channel, deadline-aware IPC, and a watchdog. Send abort immediately; if the child does not acknowledge within a short grace period, terminate the owned process/job and reset the connection. Bound startup, model loading, first-token, and total-request time separately.

Acceptance: A fake child sends no frames while staying alive. Stop finishes within the agreed deadline and the next request succeeds with a fresh connection.

F15 Cancellation is global and long file work is not fully interruptible

Priority: P1. **Evidence:** Confirmed source weakness. **Location:** desktop `AssistantState.cancel`, `send_chat_message`, `decide_action`; `hashing.rs`; `duplicates.rs`; `temp.rs::delete_approved`.

New requests reset a shared cancellation flag. Although the UI disables its input while thinking, backend calls and card actions can still overlap. Hashing loops do not check cancellation per chunk, and approved cleanup/temp mutation loops lack cancellation parameters. A large file read or long directory enumeration can therefore delay Stop or complete work after a stop request.

Fix: Use per-request IDs and cancellation tokens. Serialize mutations. Add chunk-level cancellation and time budgets to scans and hashing. Define safe operation boundaries: a started atomic move may finish, but no new operation starts after cancellation. Report partial completion explicitly.

Acceptance: Stopping a large-file scan is prompt; starting another request cannot revive the old one. A stopped batch reports completed and untouched items correctly.

F16 Model action claims are checked only when no tool was called

Priority: P1. **Evidence:** Confirmed source defect. **Location:** `services/agent/src/llm.ts:426`; `claims.ts`.

The corrective message is appended only if `callsLeft` has not decreased. Any tool call, even an unrelated calculator call, failed action, or pending-approval action, suppresses the no-action correction. A model can then falsely claim that a change was completed. The current regex also does not establish factual correctness across phrasing or languages.

Fix: Return structured tool results with operation ID, status, evidence, and approval state. Render action completion from those results. Model text may explain, but it must not supply the authoritative completion status. Verify claimed actions against actual completed operations, including failed and pending outcomes.

Acceptance: A model that calls a calculator and says it deleted a file cannot produce a success status. Waiting cards remain pending, and failed tools produce failure results.

F17 File discovery does not deliver the requested file-detail experience

Priority: P1 for the basic product. **Evidence:** Capability gap. **Location:** Rust/TypeScript tool catalogues; `find.rs` ; `reply.rs::find_summary` ; `resolve_scope` .

The live catalogue has no dedicated file-stat or folder-size tool. File search collects size but formats results using filename, parent folder name, and modification time without the size or full path. The scope is limited to six known folders and results are limited to ten. A user asking about a specific file on another drive cannot be served reliably by these primitives.

Fix: Add `get_file_info` using an opaque file ID returned by search. Report exact bytes and readable units, full location, type, modified time, and optional hash. Add read-only approved-root selection and search filters. Distinguish file length, allocated size, and folder aggregate where relevant. Do not let arbitrary model strings become filesystem paths.

Acceptance: "Find the latest invoice on D: and tell me its size" yields a correct selected result and exact bytes from the approved fixture root. Identically named candidates require explicit selection.

F18 Search scope can discard the broader root

Priority: P2. **Evidence:** Confirmed source defect. **Location:** `assistant/mod.rs:233` , `resolve_scope` .

The deduplication check skips the current path if either it is nested inside a previous path or it contains a previous path. If a child root is seen first and its parent later, the parent is dropped. Siblings under that parent are no longer searched. The comment intends to skip nested roots, but the implementation can skip the containing root.

Fix: Normalize and canonicalize roots, sort parents before children or replace contained roots when a parent is added, and deduplicate case-insensitively where appropriate. Keep root-to-display-label mapping separate from traversal.

Acceptance: Supply a child root first and then its parent. Files in sibling folders remain discoverable and no file appears twice.

F19 Scan incompleteness is not fully represented

Priority: P2. **Evidence:** Confirmed source behavior. **Location:** `walk.rs`, `find.rs`, `open_file_tool`.

The walker silently skips unreadable folders and entry errors, ignores zero-byte files by default, excludes dot-prefixed/dependency folders, and bounds visits. WalkStats tracks file count and truncation but not permission-denied roots or I/O failures. Some replies surface truncation, but `open_file_tool` discards scan statistics. Cancellation is checked outside the per-directory enumeration loop.

Fix: Return structured coverage: searched roots, excluded roots, unreadable counts, timeout/cancellation, and result truncation. Make zero-byte search behavior explicit. Use conservative cleanup exclusions but separately allow read-only discovery of approved hidden folders when needed. Never present a partial scan as exhaustive.

Acceptance: Permission-denied and budget-limited scans state that they were incomplete. A zero-byte fixture is findable. Large unreadable/directory-heavy trees remain interruptible.

F20 Repeated scans and duplicate progress undermine optimization

Priority: P2. **Evidence:** Confirmed source limitations. **Location:** `walk.rs`, `find.rs`, `duplicates.rs`, `indexer.rs`.

User queries repeatedly walk folders instead of using a persistent metadata index. Duplicate detection sensibly groups by size and prefix hash, but comparison `total` is a count of candidate files while `done` is incremented per size group. The progress fraction is not describing the same unit. Hash failures are mostly omitted rather than counted as unreadable comparison candidates.

Fix: Build a local metadata index with watcher updates and reconciliation, cache hashes by volume/file ID plus validated change indicators, and bound hash concurrency by disk type. Preserve the existing size/prefix optimization. Track scan and hashing progress in consistent units and expose skipped counts.

Acceptance: Repeated metadata queries avoid full rescans; editing a file invalidates cached metadata/hash; progress reaches a truthful completed state. Benchmarks include HDD, SSD, large folders, and many small files.

F21 Same-volume and cross-volume undo do not preserve equivalent metadata

Priority: P1 for safe file management. **Evidence:** Confirmed implementation gap; metadata loss varies by filesystem. **Location:** `fsops.rs::move_no_replace`; `cleanup.rs::copy_verified`.

Cross-volume fallback uses `io::copy` into a newly created file and verifies only the default byte stream. It does not explicitly preserve or verify Windows ACLs, alternate data streams, attributes, creation time, sparse/compressed/encrypted state, or hardlink relationships. One copy helper attempts to preserve

modified time; the move fallback does not. Equal ordinary-byte hashes therefore do not prove complete restoration of a Windows file.

Fix: Define supported file types and preservation requirements. Use appropriate Windows copy/backup APIs and verify required metadata, or refuse unsupported special files. Journal metadata snapshots and handle Mark-of-the-Web deliberately; do not strip security metadata unintentionally.

Acceptance: Cross-volume fixtures with alternate streams, restrictive permissions, attributes, and timestamps meet the supported restoration contract. Unsupported objects are skipped with a precise reason.

F22 Quarantine can worsen low space on the system drive

Priority: P2. **Evidence:** Confirmed source architecture limitation. **Location:** desktop `quarantine`, cleanup cross-volume fallback.

All desktop payloads are stored under local app data, normally on C:. Cleaning duplicates from D: may require copying their bytes onto C:. Cleaning on C: by rename does not free space until purge. There is no visible plan-level destination-space reserve/preflight. A low-space user may receive a plan that consumes scarce system-drive capacity or fails partway.

Fix: Show net capacity changes per volume and distinguish moved bytes from freed bytes. Prefer a protected vault on the source volume where feasible, with explicit policy. Reserve space for journal and rollback before running a batch; refuse insufficient-space plans.

Acceptance: A D:-to-C: plan states its C: cost. Insufficient space produces a refusal before source mutation. Same-volume quarantine is never described as immediate space recovery.

F23 Settings are changed before undo persistence and without read-back

Priority: P1. **Evidence:** Confirmed source defect. **Location:** `assistant/mod.rs:484`, `apply_setting_change`; `custodian-winsec/src/settings.rs::apply`.

The setting is written first, then history is loaded and saved. If saving fails or the process crashes, the user can be left with a change without durable recovery information. The success response follows the registry write result without a post-write read-back or proof that the desktop effect occurred. A card may describe the current value at one time and apply against a different value later.

Fix: Persist an exact typed snapshot and intended value before writing. Check the approved expected value at execution time, write through the supported adapter, read back, and record a verified outcome. Treat delayed visual effects separately from persistence. Roll back or enter a recoverable state on a persistence failure.

Acceptance: Inject storage failure and restart during a settings transaction. The old value remains known and recoverable. A concurrent external edit invalidates or refreshes the plan.

F24 Settings undo can overwrite newer user choices or lose exact values

Priority: P1. **Evidence:** Confirmed source defects. **Location:** settings `plan`, `current`, `revert`; desktop `revert_setting_tool`; `assistant/settings.rs::prune`.

History stores the previous named choice, not the raw registry value and type. An unknown existing DWORD or a read failure can become `from: None`, conflating an unrecognized/read-failed value with absence. Revert writes/removes it without checking whether the user changed the setting afterward. The history pruning code drops the oldest entries when above 200, including open changes, despite the comment saying open changes should remain.

Fix: Store raw value/type/existence and explicit read errors. Use compare-and-swap semantics for revert: only undo automatically if the current state matches what Custodian wrote. Retain unreverted recovery records or explicitly archive them with continued access. Show a settings history selector rather than a single vague "last" target.

Acceptance: Unknown DWORDs and missing values round-trip exactly. External edits are preserved. More than 200 unreverted changes retain their recovery data according to a documented retention policy.

F25 Settings coverage and conversational follow-through are narrow

Priority: P2. **Evidence:** Capability gap. **Location:** settings catalogue and value-word resolver.

Six appearance/file-view settings do not cover sound, brightness, Wi-Fi, Bluetooth, power plans, startup applications, display scaling, or machine-wide changes. Rule-based ambiguity questions do not themselves keep a typed pending clarification state; a later answer can fall through to model conversation without a deterministic binding to the proposed setting or selected file.

Fix: Implement one OS capability at a time behind read/plan/apply/verify/revert interfaces. Keep a typed pending clarification with candidate IDs, timeout, and allowed answers. Escalate only the specific privileged operation through a narrow helper; keep the model unprivileged.

Acceptance: A question such as "which dark mode?" followed by "apps" selects the stored candidate correctly. Unsupported settings are identified accurately. Each new setting has OS-version and rollback tests.

F26 Model loading and GPU behavior are tied to the development machine

Priority: P2. **Evidence:** Confirmed source behavior and validation gap. **Location:** desktop `AgentProcess::spawn`; `services/agent/src/llm.ts`.

The library defaults to GPU auto detection, but the desktop normal launch sets Vulkan when no GPU variable is supplied and disables the AMD switchable-graphics layer. This diverges from the README's general first-working-GPU description. GPU detection uses Promise.race; timing out does not by itself cancel the native operation that lost the race. Startup still lacks end-to-end deadlines and memory policy calibrated across target PCs.

Fix: Use hardware discovery and a tested capability policy. Make hardware-specific workarounds conditional and visible in diagnostics. Run backend probing in an owned subprocess with a real termination deadline. Benchmark CPU and GPU startup separately; keep known working fallback profiles.

Acceptance: CPU-only, NVIDIA, AMD, Intel, hybrid, low-RAM, broken-driver, and insufficient-VRAM scenarios either work within their budgets or report a precise unavailable state.

F27 The evaluation harness cannot block a bad release

Priority: P1. **Evidence:** Confirmed source defect. **Location:** `services/agent/eval/run.mjs`, final `process.exit(0)`; `.github/workflows/ci.yml`.

The harness prints failures but exits successfully even when cases fail. It is not invoked in CI. It resets conversation between cases, uses canned desktop responses, accepts substring matches for some arguments, and checks a narrow deletion-claim regex. This does not exercise rule parsing, real IPC/tool execution, approval state, multi-turn clarification, or filesystem results.

Fix: Add explicit score gates and a failing exit code. Save versioned structured results. Separate planner evaluation from real-tool acceptance and safety tests. Include multilingual/paraphrased requests, ambiguous targets, conflicting instructions, pending approvals, cancellation, and multiple-step tasks. Run a practical model/hardware regression job before releases.

Acceptance: Intentionally wrong outputs fail the gate. Success rates and latency are reproducible for the exact model, prompt, tool catalogue, and hardware profile.

F28 The installer does not contain a complete offline runtime

Priority: P1. **Evidence:** Confirmed configuration gap. **Location:** `apps/desktop/src-tauri/tauri.conf.json`; desktop `agent_entry`, `models_dir`, `ensure_agent`; HONESTY.

Bundle configuration declares icons but no agent/runtime/model resources. The desktop searches for a bundled server then falls back to a compile-time repository location; it launches `node` from PATH unless overridden. A development checkout can work while an installed copy on a clean PC cannot answer open-ended requests. Native node-llama-cpp libraries and GPU dependencies also require packaging and testing.

Fix: Produce a self-contained runtime package with trusted absolute paths, agent distribution, compatible Node/native libraries, and model provisioning. Distinguish an offline installation package from an optional online model-download setup. Include checksums, licenses, repair diagnostics, and version compatibility.

Acceptance: Install on a clean supported PC without Node, Rust, Python, or repository files. Disconnect the network. Start, ask supported tasks, load the model, restart, undo, and uninstall successfully.

F29 Threat protection helpers report safety without security evidence

Priority: P1 before exposure. **Evidence:** Confirmed placeholder behavior. **Location:** `downloads.rs`, `install_lab_verify.rs`, `backup_check.rs`, `install_lab.rs`.

Download verification flags the word “eicar” in the filename, not malicious content. Publisher verification, backup detection, and sandbox availability return true. An untrusted installer or missing backup could therefore be treated as verified if these helpers are connected. A test passing against this behavior proves the placeholder, not the intended protection.

Fix: Replace booleans with `verified`, `not_verified`, `unavailable`, and `error` outcomes with evidence. Integrate supported scanner/trust APIs. Validate expected publisher and signature policy separately from malware scanning. Detect a usable backup through an actual recovery test or clearly state that verification is unavailable. Fail closed for install/mutation gates.

Acceptance: Renaming a malicious test fixture does not make it pass. Missing scanner, expired/untrusted signature, wrong publisher, and unavailable backup are all reported distinctly and never treated as safe.

F30 Laya metadata decisions are not an antivirus system

Priority: P1 for security claims; P3 for model research. **Evidence:** Capability and validation gap. **Location:** `services/mlhost/mlhost/laya.py`; `training/`; `safety.py`; `tabpfn_models.py`.

The Laya prompt uses filename, AV-hit count, and a signed flag to request approve/delete/quarantine. That does not inspect executable behavior or establish whether the inputs are trustworthy. The safety helper always escalates; anomaly detection returns false; forecasting returns a fixed date. No held-out model evaluation is established by the passing IPC tests, and a Laya model was not present among the inspected model filenames.

Fix: Treat model output as an advisory signal with provenance and uncertainty. Obtain real scanner/trust signals through deterministic adapters. Remove unsupported forecasts. Require evidence and human approval before handling suspicious files; never let metadata-only model output directly authorize deletion. Evaluate false negatives and false positives on appropriate held-out data before any protective claim.

Acceptance: Unknown/missing evidence stays unknown. Prompt-injection-like filenames do not grant authority. The security result identifies the scanner evidence and distinguishes a recommendation from an executed action.

F31 Sandbox and installation helpers can claim actions that never happened

Priority: P1 before use. **Evidence:** Confirmed capability gaps. **Location:** old `spawn_in_appcontainer` in `winsec/lib.rs` ; `sandbox_factory.rs` ; sandbox `main.rs` / `templates.rs` ; `custodian-elevate` ; install recipe/undo/uninstall helpers.

The older AppContainer function launches a normal child. Sandbox `capsprobe` prints "Sandbox enforced" without probing enforcement. Elevation validates a verb and prints success but performs no privileged work. Recipes are not persisted; install undo and uninstall preparation return success without doing anything. This risks confusing future integration and reviewers because API names and successful exit codes imply functioning controls.

Fix: Disable or rename simulation APIs, return explicit unsupported errors, and consolidate on verified process-launch primitives. Implement real probes and narrow privileged actions before enabling the workflows. An installer rollback claim needs measured recovery of supported effects, not a no-op return.

Acceptance: Placeholder entry points cannot emit production success. Real probes attempt denied operations inside the child and verify observed results.

F32 Broker controls are disconnected and API endpoints can redirect credentials

Priority: P1 before any online release. **Evidence:** Confirmed source flaw, dormant in the desktop. **Location:** `services/broker/src/api.ts` , `index.ts` , `interception.ts` , `dlp.ts` , `budgets.ts` .

`new URL(req.endpoint, this.apiUrl)` accepts an absolute alternate-host URL; the request then sends the configured bearer key to that hostname. There is no same-origin restriction at this API boundary. Response data grows without a byte limit, query strings are omitted because only `url.pathname` is sent, and the timeout is not a complete response deadline. DLP is case-sensitive substring checking, network interception allows arbitrary HTTPS, and these helpers are not an enforced common gateway. Browser actions return "executed" without a browser.

Fix: Permit only validated relative paths or an explicit same-origin allow-list; reject alternate schemes, userinfo, disallowed ports, private destinations where relevant, and unexpected redirects. Preserve validated query parameters. Enforce request/response byte limits, total deadlines, consent, data classification, DLP policy, and budgets at one boundary. Return unsupported for browser actions until implemented.

Acceptance: An absolute endpoint for another host is rejected before any socket opens or key is transmitted. Slow/oversized responses terminate. Every outgoing request carries a scoped policy decision and a receipt.

F33 Persistence and orchestration add deployment complexity without serving the desktop

Priority: P2. **Evidence:** Confirmed architecture gap. **Location:** `custodian-store` , migrations 003/004/005, orchestrator activities and workflows.

The store requires PostgreSQL with pgvector and TimescaleDB. The worker requires Temporal, but desktop initialization does not provision these services. Document chunks include plain content in a database table; a future content index needs its own privacy controls. Role migrations provide selected grants but are not proof of complete least-privilege schema defaults or isolation in a real deployment.

Fix: Make a deliberate local-storage decision. Proposed: an embedded transaction/metadata store for the single-PC offline product, with protected sensitive records; keep PostgreSQL/Temporal optional only for capabilities that justify them. If retaining the current design, supply offline installers, service lifecycle management, role enforcement, extension compatibility, and restart tests. Do not run both stores without a clear source of truth.

Acceptance: A clean install provisions every required service or needs none. Startup recovery works without network. Migration and role tests run against the real supported database versions.

F34 Parsing and semantic search are neither safe nor complete yet

Priority: P2. **Evidence:** Capability gaps and source risk. **Location:** `custodian-parser/src/main.rs` ; `pipeline.rs` ; `services/agent/src/reader.ts` ; ML host `extract_embeddings` .

The parser reads an entire file, treats binary formats as lossy UTF-8, and labels type using extension. It has no file-size limit in its own entry point. The pipeline does not invoke it. Embeddings are all zeros; the reader returns a fixed “extracted” summary and detects only two English injection phrases. These pieces cannot currently provide meaningful document comprehension or semantic search.

Fix: Return unsupported for zero-vector or fake extraction endpoints. Add parsers for an explicit format set with size/page/time limits and no execution of macros or embedded programs. Run parsing in actual containment. Store content only for allowed roots, with retention/deletion controls. Use a real local

embedding model and schema dimension/version migration. Treat every document excerpt as untrusted data regardless of keyword detection.

Acceptance: Unsupported/binary/oversized files fail safely. Supported formats extract meaningful bounded text. Sensitive excluded roots never enter the content index. Semantic results improve over filename search on an evaluated fixture set.

F35 History and background tasks do not support a dependable daily assistant

Priority: P2. **Evidence:** Confirmed capability gaps. **Location:** `get_ledger` at desktop line 1920; `CleanupHistory`; reminder and safe-place keeper threads; `voice.py`.

The history UI reads cleanup manifests, not all ledger events, so settings and temp cleanup are not represented as a complete action history. Reminders and expiry work run only while the desktop is running; missed reminders are handled on next start. Voice functions return fixed or empty values. The user does not yet have a durable unified job/status view for what is pending, completed, partially failed, or recoverable.

Fix: Provide one task/action history with operation IDs and verified status, plus a recovery view. Define startup, sleep/resume, timezone-change, missed-reminder, and retention behavior. Use a small background scheduler or supported OS scheduling if reminders must fire with the UI closed. Add voice only with offline microphone permissions, push-to-talk/default privacy controls, and reliable stop behavior.

Acceptance: Every changed setting, cleanup, deletion, restore, and failure appears in history. Reminder timing survives restart/resume under the documented contract. Voice unavailable is reported truthfully.

F36 Approval views do not expose every planned item or bind all actions consistently

Priority: P1. **Evidence:** Confirmed source limitations. **Location:** desktop `MAX_VIEW_OPERATIONS = 200`, plan construction, pending actions, and `needs_card`.

Cleanup can plan up to 1,000 duplicate moves and temp cleanup up to 50,000 items, while the view includes only the first 200. Some truncation notices exist, but the user still needs an inspectable/exportable complete set to approve exact effects. Settings/action cards store a ToolCall, not a full immutable expected-state plan. Known-app/folder calls and task mutations do not all use the same card/signature policy; this may be intentional low-risk behavior but is not expressed through one auditable risk table.

Fix: Keep quick actions for genuinely low-risk operations, but define them centrally. Provide the complete affected-item list, totals, exclusions, irreversible effects, expiry, and target identities. Bind medium/high-risk consent to a canonical plan including old/new state and intended recipients. Do not indiscriminately

add prompts to every read-only operation.

Acceptance: A large plan lets the user inspect every item before approving. Changed targets invalidate stale consent. Task/app/settings/file actions follow a documented risk policy.

F37 Journal size limits are checked only on opening

Priority: P2. **Evidence:** Confirmed source defect. **Location:** `crates/custodian-journal/src/wal.rs`, `Journal::new` versus `write_record`.

`MAX_JOURNAL_BYTES` is enforced in the constructor, but append does not check current length plus the new record. An already open journal can exceed its stated 64 MB cap, fill disk, and later refuse to reopen. Large unbounded records also increase recovery memory use.

Fix: Check maximum record size and projected journal size before appending. Introduce safe segment rotation/checkpointing without dropping required recovery records. Reserve capacity for completion/recovery events.

Acceptance: Appends crossing the limit fail before writing any bytes; reopening remains possible; required active-transaction recovery survives rotation.

F38 Replay prevention and wire schemas are not yet a complete process contract

Priority: P2 before separating execution services. **Evidence:** Confirmed source design gap. **Location:** `custodian-approval::Verifier`; `custodian-proto`; desktop/agent request definitions.

Consumed approval nonces are remembered only in memory. That is adequate for the current short-lived in-process route only if old approvals cannot re-enter after restart; it is not a durable exactly-once execution guarantee for a future IPC engine. Transaction wire `SignedApproval` contains just ID/signature, while the live approval object includes nonce and expiry. Engine kinds and generic wire kinds also differ. IPC frame authentication is valuable, but does not solve semantic authorization or transaction idempotency.

Fix: Define versioned operation/request/result/approval schemas, canonical plan encoding, correlation IDs, protocol version negotiation, and persistent execution IDs. Consume approvals and commit transaction state durably as one coordinator operation. Generate mirrors and test cross-language compatibility, including rejection of unknown/invalid fields where required.

Acceptance: A valid previously consumed approval cannot execute again after restart. Rust, TypeScript, and Python agree on exact protocol examples and reject incompatible versions safely.

F39 Documentation and release verification can overstate readiness

Priority: P2. **Evidence:** Confirmed contradictions and test limitations. **Location:** README, HONESTY, ADVANCE, CI, and fixture-gated tests.

The README claims all changes are sandbox-tested and describes default agent isolation that the live path does not enforce. ADVANCE includes older architecture snapshots. Cross-volume tests return early when no second-volume variable is provided, so a green suite does not necessarily test cross-volume moves. Some tests deliberately confirm stub success. No observed clean-machine release gate tests the installed product.

Fix: Keep one capability-status registry that drives docs and runtime help. Separate simulated tests from real implementation tests and mark skipped prerequisites explicitly. Block releases on supported-platform end-to-end, containment, recovery, installation, and model gates. Update readiness claims from measured evidence.

Acceptance: Documentation agrees with default behavior. Test summaries show skipped essential scenarios. A release cannot pass solely through stub returns or frontend compilation.

7 Recommended target architecture

The following is a proposed consolidation, not a statement that these components already exist.

```
Desktop UI
  | requests and user decisions
  v
Request coordinator and clarification state
  | rule parser or isolated local model proposes typed steps
  v
Capability registry and access policy
  | roots, private data classes, offline policy, risk tier
  v
Plan builder and approval service
  | exact identities, expected state, effects, recovery contract
  v
Single transaction executor
  | durable intent -> controlled operation -> verification -> completion
  +--> filesystem adapters
  +--> settings adapters
  +--> application adapters
  +--> narrow elevation helper when required
  v
Protected local state, history, recovery and diagnostics
```

The model remains an isolated planner/explainer. It sees only the metadata or bounded content that the policy authorizes. It never receives direct unrestricted filesystem, registry, network, or shell access. A model error must become a rejected plan, a clarification, or a visible proposal rather than a silent destructive action.

7.1 Typed action contract

Every state-changing adapter should implement:

Stage	Required information or behavior
Capability	Availability, supported OS/app versions, required privilege, offline suitability
Read	Accurate current state with explicit missing/error outcomes
Plan	Exact targets and identities, expected state, effects, risk, estimated cost, undo availability
Authorize	Policy decision and consent bound to the immutable plan
Prepare	Durable recovery intent and required capacity reserved before changes
Execute	Restricted API calls; no arbitrary model-authored command text

Stage	Required information or behavior
Verify	Actual state/content/identity checked against the intended result
Complete	Durable verified result and user-visible operation status
Revert	Conflict-aware rollback when supported; precise irreversible disclosure otherwise

7.2 Access and risk policy

Use separate controls for metadata discovery, content reading, modification, and external transmission. A user may allow filename/size search in Documents while denying content reading of a private folder. Excluded roots must apply to search, model results, indexing, recovery, and network payloads consistently.

Suggested risk tiers:

Tier	Examples	Consent and enforcement
Read-only	Size, free disk space, metadata in approved roots	Fast execution; access policy enforced
Low-impact local	Open a known folder, add a task	Clear request intent and documented quick-action policy
Reversible modification	Rename/move files, change supported settings	Exact plan/card; durable recovery; verified result
Irreversible or security-sensitive	Purge, delete, install, private-content export	Explicit effects and stronger native consent; deny if safeguards unavailable
Privileged	Machine-wide supported setting	Narrow elevation helper; specific authenticated plan; no unrestricted agent privilege

Offline mode should enforce local execution. Any optional network exception must identify the recipient, data, duration, and action scope. Do not add cloud inference as an implicit fallback when local inference fails.

8 Prioritized implementation programme

These are dependency-ordered work packages. Duration depends on team size and supported task breadth; there is insufficient evidence for a reliable calendar estimate.

Phase 1 Stabilize truth and safety

Work: Correct default sandbox behavior; remove unrestricted release fallback; enforce essential jobs; disable destructive/success-reporting stubs; implement central offline checks; return truthful action status; fix missing-ledger detection; document current capability limits.

Deliverables: An accurately labelled prototype with deterministic tasks still available when AI is unavailable; no dangerous stub reachable through a public production path.

Exit gate: Default containment tests pass. Unsupported actions cannot alter fixtures or claim success. No network payload leaves through an unapproved route.

Phase 2 Build one recoverable mutation route

Work: Choose the storage/coordinator design; create durable intent/completion state; authenticate manifests; enforce root/file identity; serialize mutations; implement exact settings snapshots; fix same-volume undo and cloud purge behavior; define metadata preservation and capacity preflight.

Deliverables: Cleanup and settings use the same transaction contract. Startup offers recovery for interrupted operations. History shows verified results and partial failures.

Exit gate: Crash injection, disk-full, corruption, concurrency, and reparse-race fixture tests pass. No orphaned payload or unrecoverable unrecorded settings change remains after supported failures.

Phase 3 Complete the core offline file assistant

Work: Approved roots; complete metadata results; selected-file IDs; file/folder size tools; multi-turn candidate selection; indexed name/filter search; copy/move/rename/create adapters; staged cleanup with explicit space accounting.

Deliverables: The user's basic file-assistant examples work without network or a model for familiar task forms. Open-ended requests can use the local model through the same validated tools.

Exit gate: End-to-end fixture tasks verify correct targets, exact sizes, approved effects, and undo. Ambiguous requests never choose a destructive target silently.

Phase 4 Expand settings and application capabilities

Work: Add typed adapters for chosen priorities such as volume, brightness, power options, and supported app tasks. Implement privilege separation where needed. Add OS-version checks, external-edit conflict handling, and supported scheduler behavior.

Deliverables: A clearly published capability catalogue with tested behavior and per-action recovery limits.

Exit gate: Each adapter has read/plan/execute/verify tests and real-desktop acceptance. Unsupported hardware/settings return unavailable rather than success.

Phase 5 Ship a complete offline package

Work: Bundle runtime/agent/native libraries; provide offline model installation; implement diagnostics; run CPU/GPU profiles; add release gates; test repair/update/uninstall and state migration.

Deliverables: A versioned installer that functions without a checkout or developer tools.

Exit gate: Clean supported PCs complete the offline acceptance suite. Required services start and stop cleanly, no model download occurs without consent, and runtime assets verify before loading.

Phase 6 Add document understanding and protective integrations

Work: Safe bounded parsing; content-access policy; real local embeddings; scanner/trust evidence; private-index lifecycle; optional voice. Online browser automation is a separate optional phase after offline reliability.

Deliverables: Useful document retrieval and transparent security advice based on deterministic evidence.

Exit gate: Parsing/injection/adversarial-content tests pass. Security recommendations identify their evidence and uncertainty. No model output independently grants delete/install authority.

9 Acceptance tests for the actual product

Run filesystem cases in marker-protected fixtures and settings cases in a disposable Windows VM or dedicated test profile. Never use personal documents as destructive test inputs. Tests below are proposed completion criteria, not claims that they passed during this audit.

ID	Scenario	Required outcome
A01	Install without developer tools	Offline runtime starts using bundled assets

ID	Scenario	Required outcome
A02	Start with Wi-Fi off	Supported local tasks work; no cloud fallback
A03	No model installed	Clear model-unavailable result; deterministic tools work
A04	Model hash mismatch	Model is refused before native parsing
A05	Sandbox setup failure	No unrestricted agent child starts
A06	Agent probes private root	Read denied; no content leak into output
A07	Agent probes network	Network denied in default offline mode
A08	Job assignment failure	Child stopped; resource state reported accurately
A09	Find file in Documents	Full path, size, and modified time match fixture
A10	Find file in approved D: root	Correct target found; unapproved roots excluded
A11	Duplicate filename candidates	User chooses by stable candidate ID
A12	Query file size	Exact bytes and human-readable size agree
A13	Query folder size	Scope, skipped entries, logical/allocated meaning disclosed
A14	Search empty file	Zero-byte file returned
A15	Search inaccessible folder	Partial coverage disclosed
A16	Root child before parent	Parent siblings searched; no duplicate results
A17	Clarification response	Follow-up binds to the pending task
A18	Large scan Stop	Stops within measured deadline
A19	Silent agent Stop	Abort/termination works without waiting for output
A20	Two overlapping requests	Independent IDs/tokens; no cross-cancellation
A21	Duplicate plan changed after approval	Changed source/keeper skipped or plan invalidated
A22	Move/rename destination exists	Existing file never overwritten
A23	Same-volume quarantine undo	Original bytes and supported metadata restored
A24	Stored payload modified	Undo refuses integrity failure

ID	Scenario	Required outcome
A25	Cross-volume move	Required streams/attributes/permissions preserved
A26	Cross-volume capacity insufficient	Refuse before source mutation
A27	Kill between intent and mutation	Restart correctly reconciles pending state
A28	Kill after move before completion	Payload remains discoverable and recoverable
A29	Kill during undo	Restart reports actual restoration state
A30	Kill during purge	No missing/unclassified deletion state
A31	Corrupt manifest or inject outside path	Refuse without following attacker-supplied paths
A32	Replace parent with junction	No operation escapes approved roots
A33	Keeper becomes cloud placeholder	Verified local copy retained
A34	Purge and undo overlap	Serialized consistent outcome
A35	Ledger deleted while anchor remains	Integrity alert; risky mutations blocked
A36	Ledger/anchor write fails	Recoverable state; no fabricated success
A37	Settings storage unavailable	No unrecorded write proceeds
A38	Change then externally edit setting	Undo does not overwrite the external choice
A39	Unknown raw registry value	Exact typed value survives change/revert
A40	More than 200 unreverted changes	Recovery data retained under policy
A41	Model claims success after failed tool	UI shows actual failure
A42	Model claims success while card pending	Pending state remains explicit
A43	Model calls unrelated tool and claims action	False action completion suppressed
A44	Open unrecognized/program-like file	Policy refusal or safe handoff
A45	Reminder during sleep/restart	Matches documented missed-reminder policy
A46	Unsupported stub API invoked	Unsupported outcome; no mutation
A47	Suspicious file renamed	Scanner result independent of filename

ID	Scenario	Required outcome
A48	Signature/scanner unavailable	Unknown/unavailable outcome, not verified
A49	Malformed/oversized document	Parser contained; bounded resource use
A50	Private folder indexed/excluded	Exclusion honored; prior content removable
A51	Optional API alternate-host endpoint	Rejected before credential transmission
A52	Oversized/slow optional response	Byte cap and absolute deadline enforced
A53	Replay approval after restart	Already-executed transaction rejected
A54	Journal limit exceeded	Refuse append before bytes written; reopen succeeds
A55	Uninstall/upgrade interrupted	User files and supported recovery records preserved

10 Performance and accuracy targets

The user wants accurate, timely, optimized execution. Measure these separately; a low idle CPU figure does not establish fast file search or reliable action selection. The following are proposed release targets and need calibration on declared minimum hardware.

Metric	Proposed initial gate	Measurement
Warm indexed metadata query	p95 under 1 second on target fixture	End-to-end request to verified result
Ordinary deterministic status task	p95 under 2 seconds	UI to result, excluding deliberate sampling time
Stop acknowledgment	Under 1 second; owned-child termination after short grace	Silent child and large-file scan cases
Warm model first token	Publish measured p50/p95 by CPU/GPU profile	No unrelated workload; pinned model/prompt
Model load and total reply	Publish upper budgets with timeout behavior	Cold cache and warm cache separately
Plan selection	At least 95 percent on representative held-out suite	Correct target, tool, parameters, clarification

Metric	Proposed initial gate	Measurement
Executed unsafe/wrong target	Zero in release acceptance suite	Filesystem/OS postconditions, not model prose
Recovery	Every injected supported crash classified and recoverable	Boundary-by-boundary failure tests
Private data egress	Zero unauthorized transmissions in tested paths	Network capture and recipient/payload assertions
Idle resources	Published process-tree CPU/RAM budgets	UI, WebView, agent, scheduler, optional services

“Zero observed wrong actions” in a finite suite is a release gate, not a universal guarantee. Reliability comes from refusing unsafe or ambiguous work, exact verification, containment, and recovery as well as model accuracy.

Optimize in this order: avoid unnecessary inference with well-tested rules; avoid repeated scans with an index; cache validated hashes; limit disk concurrency; use bounded excerpts instead of dumping content into the model; calibrate model profiles; optimize animation only when it is a measured resource bottleneck.

11 Build and release remediation

1. Reproduce Rust build failures in an ordinary developer shell with the same working tree. The review sandbox reports denied canonicalization on D:, so do not treat those failures as conclusive product-code bugs. Record exact Rust/Node/Python versions and use locked dependencies.
2. For `OUT_DIR`, verify build-script execution with verbose Cargo logs and a fresh **task-specific target directory**, ensure the frontend exists, and check the Tauri build configuration. The repository already has `build.rs`; adding a second build script without diagnosis is not a sound fix.
3. For `sqlx::migrate!`, confirm canonical access to the migrations directory under the build token and run store/engine tests where the supported database prerequisites are available. A test compilation blocked by sandbox access says nothing about the correctness of the migrations.
4. Stabilize JavaScript temporary directories in restricted Windows environments when needed. The workspace-local rerun passed; the first `ENOENT` result should remain a test-environment issue unless it reproduces in a normal supported setup.
5. Add a real installer build and clean-machine smoke job. The frontend bundle is only one component. Include runtime/model/native-library validation and default AppContainer probes.

6. Make model evaluation exit nonzero when below its threshold. Publish machine-readable results and link the tested model/prompt/catalogue versions to each release.
7. Add required second-volume coverage and indicate skipped tests clearly. Keep simulation/stub checks separate from production feature acceptance.
8. Execute dependency/security audits against available official advisory feeds before release. Existing installed dependencies and a passing source scanner do not establish current vulnerability status. This review did not run fresh `npm audit`, `pip-audit`, or `cargo deny` advisory checks.

12 Source map for follow-up implementation

Area	Principal source
Live request handling, cards, approval, background tasks	`apps/desktop/src-tauri/src/assistant/mod.rs`
Action-card descriptions and expiry	`apps/desktop/src-tauri/src/assistant/actions.rs`
Deterministic intent and Rust tool validation	`assistant/intent.rs`, `assistant/tools.rs`
Agent child lifecycle and blocking IPC	`apps/desktop/src-tauri/src/agent.rs`
Local model loading, generation, claim checks	`services/agent/src/llm.ts`, `claims.ts`, `salvage.ts`
Agent framed server and tool catalogue	`services/agent/src/server.ts`, `tools.ts`, `ipc.ts`
Live file cleanup and metadata manifests	`crates/custodian-core/src/cleanup.rs`
Search, walking, hashing, temp deletion	core `find.rs`, `walk.rs`, `duplicates.rs`, `hashing.rs`, `temp.rs`
Windows containment and filesystem primitives	winsec `appcontainer.rs`, `job.rs`, `fsops.rs`, `lib.rs`
Settings adapter and history	winsec `settings.rs`, `registry.rs`; desktop `assistant/settings.rs`
Ledger and approval keys	desktop `history_ledger.rs`; `custodian-ledger`; `custodian-approval`
Dormant engine execution and recovery	engine `approval.rs`, `recovery.rs`, `fs.rs`, `ops/`
Journal persistence	`crates/custodian-journal/src/wal.rs`
Optional data store	`crates/custodian-store/src/`, `migrations/`

Area	Principal source
Optional network and workflows	`services/broker/src/` , `services/orchestrator/src/`
Parsing, embedding, voice, security model	`custodian-parser` ; `services/mlhost/mlhost/` ; `training/`
Installer and quality gates	Tauri config, `.github/workflows/ci.yml` , model eval harness
Current product claims and planned decisions	README, HONESTY, ADVANCE, `docs/design/`

13 External technical references

Microsoft documents AppContainer as a resource-isolation boundary. Its intended use supports keeping the model process isolated even when Rust tool arguments are already restricted. See AppContainer isolation.

Windows file access involves object security and access rights in addition to ordinary file contents. Metadata-preservation and handle-access requirements should be validated against File security and access rights.

Publisher trust should use the supported trust-verification mechanism and a clear application policy rather than a constant boolean. See WinVerifyTrust. Signature trust is one source of evidence; do not equate it with a complete malware verdict.

14 Recommended first implementation batch

The first batch should address F01, F02, F09, F11, F14, F16, F27, and the unsupported-success portions of F29–F32. These changes make the prototype's behavior truthful and contain failures without needing to finish every planned subsystem.

The next batch should build the transaction coordinator and migrate cleanup/settings through it, addressing F03–F08, F10, F21–F24, and F36. Only after those gates pass should the team expand file management, settings, semantic understanding, or privileged tasks.

The target is an assistant whose abilities can grow safely: every advertised capability has a real execution route, every action has a verified result, private data access is controlled, and failures leave a clear recoverable state.

15 Verification addendum

The full command `cargo test --workspace --offline --locked`, with a task-local temporary directory, completed successfully outside the restricted review sandbox: **263 tests passed and three were explicitly ignored**. The desktop, core, engine, Windows-security primitives, approval, IPC, journal, ledger, store unit checks, and other workspace tests all completed without test failures. The ignored tests were real-model operation under job limits, real-model operation in the sandbox, and listing this PC's Start-menu apps.

The prior default-temp rerun inside the restricted sandbox produced 30/43 passing core tests and six of seven approval tests, with filesystem operations returning zero completed actions or access-denied errors. The earlier workspace-local-temp run passed 42/43 core tests. Both sets of failures disappeared in the unrestricted full-workspace rerun; this establishes an environment dependency rather than a confirmed application regression in those tests.

The passing full-workspace run includes real AppContainer denial tests and job-object tests. These demonstrate that the security primitive can work, but do not change F01: the desktop's current default branch still bypasses that primitive unless its environment flag is `1`.

The cross-volume filesystem test was reported as passed by Rust, but its implementation returns early when `CUSTODIAN_SECOND_VOLUME` is absent. No second-volume fixture was configured for this review, so actual cross-volume movement and metadata preservation remain unverified. The store's passing unit test formats a vector literal; it does not start PostgreSQL, apply migrations to a real database, or enforce roles.

Final verification conclusion: JavaScript tests, Python tests, TypeScript checks, lint checks, the frontend build, and the Rust workspace tests pass in suitable test environments. The findings in this report mainly concern missing integration, unsafe contracts, untested failure paths, and capability gaps that the current suites do not assert.

16 Extension: original product intent and the reported daily-use failures

This extension addresses the user's reports that Custodian reuses old answers, cannot reliably identify the largest file across the system, cannot perform requested file/settings changes, and does not yet execute tasks alongside the user. It adds seven findings to the initial 39. The detailed companion is `D:\Custodian\docs\CUSTODIAN_OFFLINE_ASSISTANT_IMPLEMENTATION_ROADMAP.md`, with a readable HTML edition beside it.

The supplied originals under `C:\Users\paagu\OneDrive\Desktop\ideas` were compared against `docs/design`. The architecture, build plan and master document have matching line-by-line text after newline normalization. This is implementation drift, not a different initial vision. The original README

requires synchronized canonical decisions across those documents; the roadmap proposes changes without silently rewriting that register.

The original product is a bounded offline assistant with a single writer, exact approvals, recovery before action, evidence-backed explanations and isolated model contexts. It supports a growing set of useful tasks, but explicitly excludes unrestricted security/system changes. The user's wish for broad assistance should become a tested capability catalogue, not unrestricted model shell or permanent administrator access.

F40 Conversation state and observations can diverge from current task truth

Priority: High. **Evidence:** Confirmed source behavior and integration gap; the exact cause of the user's stale-answer incident remains untraced.

Source: `services/agent/src/llm.ts:291,380–447` ; `services/agent/src/server.ts:14–39` ; `apps/desktop/src-tauri/src/assistant/mod.rs` direct intent dispatch and `decide_action` ; `apps/desktop/src/App.tsx:251–272` .

Problem: The agent retains a `LlamaChatSession` across model-mediated turns. Although the engine exposes `reset()` , the live protocol has no explicit conversation reset/identity contract. Tool results are strings without freshness, object version or scope metadata. Directly executed recognized commands and subsequent approval outcomes are not consistently reflected in the model's context. Model idle unloading can lose context while visible UI history remains. Thus history can be both stale and incomplete.

Effect: An old size or location may be reused as current, while a follow-up such as “move that file” may lack the direct query result the user can see. A rejected or completed task may remain represented to the model only as waiting for approval. The instruction after the tool budget is exhausted to answer from existing knowledge does not enforce current evidence.

Important distinction: This finding does not prove a disk-backed long-term memory store caused the symptom. In-memory chat history, missing context, fallback text, failed tools and unsupported model inference need separate diagnosis. Erasing memory or increasing model size is not a sufficient fix.

Fix: Own task/conversation events outside the model process. Feed deterministic observations and approval/execution outcomes into the correct task context. Separate preferences, historical receipts and current observations. Include task/conversation/observation IDs, time, scope and versions in validated results. Re-observe mutable PC facts and derive completion claims only from verified receipts. Add explicit reset/rebuild behavior and identity-backed follow-up references.

Acceptance: Ask a fixture file's size, change it externally, and ask again in the same conversation. Repeat after a direct query, declined approval, completed action and model restart. New answers use new evidence; historical facts are labelled; no replay causes another mutation. Roadmap B01–B08.

F41 “My system” is silently narrowed to personal folders

Priority: High for product correctness. **Evidence:** Confirmed source behavior.

Source: `apps/desktop/src-tauri/src/assistant/intent.rs:63-64,312,1004` ; `assistant/mod.rs:233,1229-1260` ; `crates/custodian-core/src/walk.rs:13-50,80-133` ; `crates/custodian-core/src/largest.rs` .

Problem: `Scope::UserFolders` explicitly interprets “my system” as the six supported personal folders. The largest-file dispatcher always asks for ten results. The walker excludes system/dependency/dot directories, skips reparse/offline files and unreadable locations, and stops at a file budget. Its response path does not fully represent those coverage limits. This extends F17–F20 with the exact scope mismatch behind the user's example.

Effect: A larger file on another local drive or outside the selected folders is not even considered. A correct top-K algorithm over a subset cannot establish the largest file in the requested whole-system scope. Logical size also does not necessarily equal allocated or reclaimable space.

Fix: Introduce explicit volume/folder scopes, requested count and size metric. Resolve authorized local volumes for whole-system queries. Separate read-only discovery policy from write/cleanup eligibility. Enumerate metadata with per-task budgets, cancellation and typed coverage statistics. Display interval, exclusions, access failures and partial status. For indexed queries, track complete baseline/change coverage; rechecking old top results alone cannot prove an exact current maximum.

Acceptance: A fixture's largest file on a second volume and outside known folders is returned under the appropriate metadata grant. Budget exhaustion and unreadable locations produce a partial result. Singular and top-N requests respect cardinality. No content reads/cloud hydration are required for logical-size discovery. Roadmap B09–B19.

F42 General file operations are absent from the live capability catalogue

Priority: High capability gap. **Evidence:** Confirmed catalogue gap; some narrower cleanup/settings operations already exist.

Source: `apps/desktop/src-tauri/src/assistant/tools.rs:19-83` ; `services/agent/src/tools.ts` ; `apps/desktop/src-tauri/src/assistant/settings.rs` ; `crates/custodian-winsec/src/settings.rs` .

Problem: The live tool enum does not implement general create/copy/move/rename/edit capabilities. Existing settings support is limited. Understanding a command in natural language does not make the corresponding execution handler exist. Opening an application/settings page is not equivalent to applying the requested change.

Effect: The assistant can discuss an operation without being able to perform it. Broadening permissions cannot add a missing operation and would expose more risk while the transaction defects remain.

Fix: Use a versioned capability registry with implemented/available/blocked status, typed parameters, preconditions, risk, preview, handler, verifier and inverse. Build the common Engine transaction first, then add bounded file and settings adapters. Both deterministic and model requests use that same route. Unsupported tasks return explicit unavailability or a labelled handoff.

Acceptance: Each advertised file operation is exercised through the installed UI with an actual fixture effect, verified receipt, failure case and undo/conflict case. Requests outside the catalogue never report completion. Roadmap B20–B28 and milestones M4–M5.

F43 Agent ping readiness does not establish actual model readiness

Priority: Medium, high for startup diagnosis. **Evidence:** Confirmed source behavior.

Source: `services/agent/src/llm.ts:432-443` ; `services/agent/src/server.ts` ping branch; desktop `apps/desktop/src-tauri/src/agent.rs` startup/health path.

Problem: `check()` checks model file readability and imports `node-llama-cpp` ; real model/session initialization occurs in `getSession()` through warm-up or chat. The pong's `ready` flag can therefore succeed without proving the model can load and answer. This distinction sharpens the readiness discussion in F01/F26; a ping must not be described as a completed model-load test.

Effect: The application can appear ready, then fail on the first actual model request. A corrupt/incompatible model, runtime initialization failure or resource failure can be misdiagnosed as bad understanding.

Fix: Separate process alive, protocol authenticated, runtime available, model loading, model ready and sandbox/capability readiness. Surface actionable failure reasons and keep independently available deterministic tasks usable. Perform a bounded actual initialization/probe before reporting model-ready.

Acceptance: A readable invalid model file can satisfy only file/runtime checks, never model-ready. First real request does not discover a previously hidden readiness failure. Roadmap B35 and R02.

F44 Interface success signals are not tied to verified outcomes

Priority: High for truthful operation. **Evidence:** Confirmed source behavior.

Source: `apps/desktop/src/App.tsx:231-265` ; backend `decide_action` text result contract; existing F16.

Problem: The frontend reacts happily after a successful `send_chat_message` invocation and after the person allows an action. Approval-card state is updated to allowed before execution returns. Allowed is a valid permission state, but the happy reaction is not conditioned on verified task success; a resolved invocation or returned text may describe a failure or no completed operation.

Effect: The product can imply success through its interaction even when no intended system change occurred. The user must interpret prose to find the real outcome, and the UI cannot reliably distinguish waiting, failed, partial and completed work.

Fix: Define typed task states and results. Keep approval, execution and verification separate. Drive animation, task cards, history and assistant completion text from the same authoritative coordinator result. Only verified completion renders success; partial/cancelled/recovery-required retain their own states.

Acceptance: Denied approval, unsupported capability, failed setting read-back, tool timeout and partial batch all produce their correct visible state. An unrelated successful tool cannot validate another action claim. Roadmap B03–B04, B08, B26, B28 and R06/R17.

F45 Current Laya decisions differ from the original escalate-only design

Priority: High design/security gap. **Evidence:** Confirmed contract mismatch; not a claim that every returned decision currently reaches an active deletion route.

Source: `services/mlhost/mlhost/laya.py` ; original Architecture canonical D-19 and Build Plan steps 105–106. Related initial findings F29–F30.

Problem: The original design specifies calibrated D1–D6 decisions that may only add caution, with deterministic-only fallback. The current classifier asks a chat model to return approve/delete/quarantine from filename, AV-hit count and signature metadata. Its interface and model/runtime design differ materially from the original planned INT8 ONNX component.

Effect: Future integration could mistake a model recommendation for authorization or malware evidence. Merely training the classifier more does not fix this authority mismatch. A name/signature summary cannot establish that content is safe or should be destroyed.

Fix: Restore an escalate-only recommendation interface enforced by deterministic policy. Model errors/unavailability abstain. Test that ML can never lower risk, remove protection or grant permission. Keep real security-provider evidence separate and report unknown state honestly. Any different model/runtime choice needs an explicit canonical design decision and measured evaluation.

Acceptance: A planted ML “approve” or “delete” result cannot override a denied/protected action or directly authorize deletion. Missing AV/provider evidence never becomes a safe verdict. Roadmap B38–B39 and R31–R32.

F46 Side-by-side work lacks an integrated durable task coordinator

Priority: High capability/integration gap. **Evidence:** Confirmed UI/runtime structure; orchestration components exist but do not establish the intended complete desktop task lifecycle.

Source: `apps/desktop/src/App.tsx` `isThinking` and `pendingPlan`; desktop agent/cancellation paths; original Architecture A13; initial F08, F14–F15, F33 and F35.

Problem: A conversation's thinking state, global cancellation and single pending-plan slot do not represent multiple durable jobs. The product needs separate task IDs, per-task stop, resource locks, progress, approval state and restart reconciliation. A reminder list is also not the original Later system, which requires a deferred plan and fresh planning when its trigger fires.

Effect: Long work can block interaction, concurrent decisions can be confusing, and restart/cancellation can lose the relationship between what was requested and what actually happened. Adding parallel threads before resource locking would worsen mutation races.

Fix: Implement a coordinator-backed task state machine and task board. Serialize planner inference if needed for memory, while scheduling bounded independent tool work. Lock conflicting file/destination resources, prioritize control messages, yield to foreground activity and keep the UI nonintrusive. Reconcile durable transactions after restart before retry. Re-plan Later work against current state and valid authority.

Acceptance: A person can ask a short question during a scan, cancel one of two tasks independently, review two distinct approvals, and restart into truthful task states. Two writes to one object cannot race. Roadmap B29–B33 and M6.

16.1 Resolution programme and validation limits

The companion roadmap specifies memory/evidence contracts, whole-volume search behavior, a prepare/authorize/execute/verify/recover transaction, file/settings adapters, task coordination, privacy and security boundaries, and offline installation. It includes milestones M0–M8, backlog R01–R38, original-step mapping and proposed acceptance tests B01–B40 in addition to this audit's A01–A55.

Do not treat this extension as a product patch or a new full test run. It documents inspected behavior and proposed fixes. The existing automated verification evidence remains exactly as qualified in section 15. The exact stale-answer incident, installed whole-PC scan behavior and proposed end-to-end acceptance cases still require controlled execution during implementation.